

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum 12 bertujuan agar mahasiswa memahami :

1. Memahami konsep *transaction control language* (TCL) dalam pengelolaan transaksi pada basis data untuk menjaga konsistensi data.
2. Mempelajari penggunaan perintah `COMMIT` untuk menyimpan perubahan data secara permanen ke dalam *database*.
3. Mempelajari penggunaan perintah `ROLLBACK` untuk membatalkan perubahan data sebelum disimpan secara permanen.
4. Memahami fungsi `SAVEPOINT` sebagai penanda dalam transaksi untuk memungkinkan kembali ke titik tertentu.
5. Menerapkan pengelolaan transaksi menggunakan TCL dalam operasi manipulasi data seperti `INSERT` dan `UPDATE` agar proses lebih terkontrol.

12.2 Dasar Teori

Transaction control language (TCL) merupakan sekumpulan perintah dalam SQL yang digunakan untuk mengelola perubahan data yang dilakukan oleh perintah *Data Manipulation Language* (DML). TCL berfungsi untuk memastikan bahwa setiap transaksi dalam *database* dapat dikontrol dengan baik agar menjaga konsistensi dan integritas data. Perintah-perintah yang termasuk dalam TCL antara lain `COMMIT`, `ROLLBACK`, `SAVEPOINT`, `SET TRANSACTION`, dan `SET CONSTRAINT`. Sebagian besar perintah TCL juga didukung dalam PL/SQL, meskipun terdapat beberapa pembatasan khusus pada penggunaan `COMMIT` dan `ROLLBACK` dalam kondisi tertentu (Oracle, 2026).

Perintah `COMMIT` digunakan untuk mengakhiri suatu transaksi dan menyimpan seluruh perubahan secara permanen ke dalam *database*, sekaligus menghapus semua `SAVEPOINT` serta melepaskan kunci transaksi yang ada. Sebaliknya, perintah `ROLLBACK` digunakan untuk membatalkan perubahan yang telah dilakukan dalam suatu transaksi dan mengembalikan data ke kondisi sebelumnya tanpa memerlukan *privilege* khusus, kecuali pada transaksi

terdistribusi tertentu yang memerlukan hak akses khusus seperti `FORCE TRANSACTION`. Sementara itu, `SAVEPOINT` digunakan untuk membuat titik acuan dalam transaksi sehingga pengguna dapat melakukan `ROLLBACK` ke titik tertentu tanpa membatalkan seluruh transaksi. Dengan adanya fitur-fitur ini, pengelolaan transaksi menjadi lebih fleksibel, aman, dan terkontrol (Oracle, 2026).

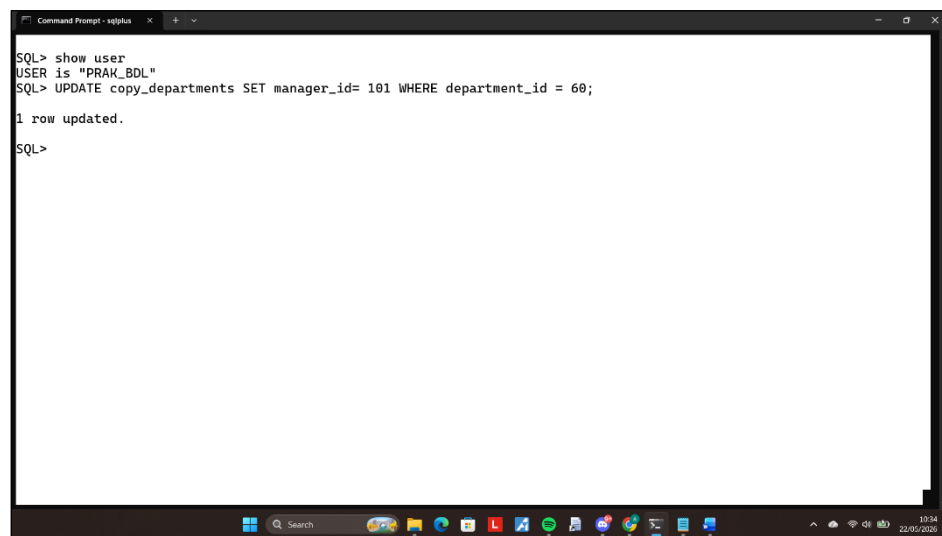
12.3 Hasil dan Pembahasan

Hasil dan pembahasan yang telah dilakukan pada praktikum 12 adalah sebagai berikut.

1. *Query*:

```
UPDATE copy_departments SET manager_id= 101 WHERE  
department_id = 60;
```

Hasil:



Gambar 12.1 Perubahan manager pada tabel `copy_departments` berdasarkan kondisi tertentu

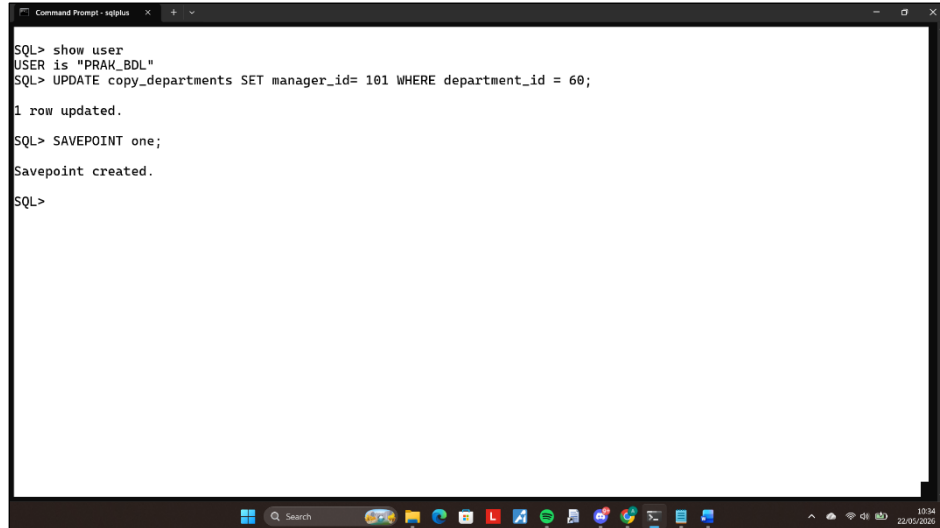
Penjelasan:

Query ini digunakan untuk memperbarui data pada tabel `copy_departments` dengan mengubah nilai `manager_id` menjadi 101. Perubahan dilakukan hanya pada baris yang memenuhi kondisi `department_id = 60` sehingga *update* bersifat selektif.

2. *Query*:

```
SAVEPOINT one;
```

Hasil:



```
SQL> show user
USER is "PRAK_BDL"
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;
1 row updated.
SQL> SAVEPOINT one;
Savepoint created.
SQL>
```

Gambar 12.2 Pembuatan `SAVEPOINT` sebagai penanda transaksi pada *database*

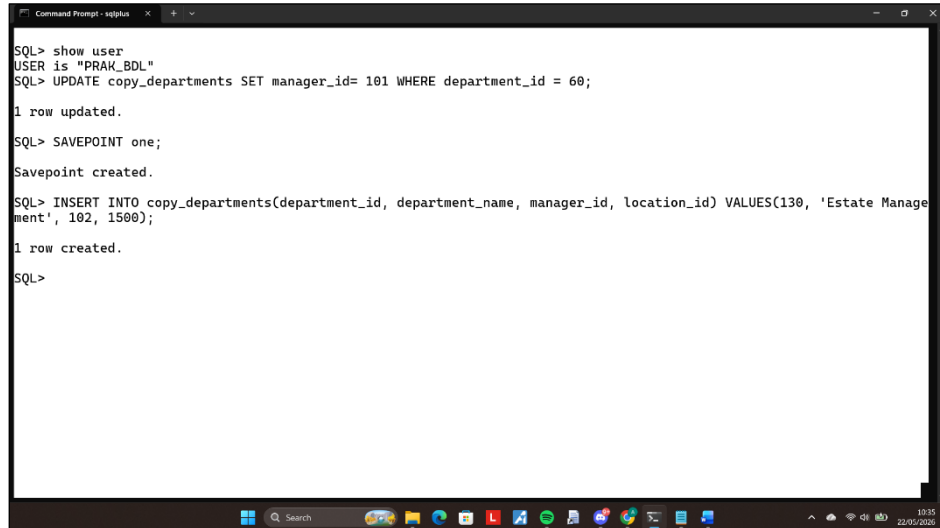
Penjelasan:

Query ini digunakan untuk membuat `SAVEPOINT` dengan nama `one` sebagai titik acuan dalam suatu transaksi. `SAVEPOINT` memungkinkan pengguna untuk kembali ke kondisi tertentu tanpa membatalkan seluruh transaksi yang telah dilakukan.

3. *Query*:

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:



```
SQL> show user  
USER is "PRAK_BDL"  
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;  
  
1 row updated.  
  
SQL> SAVEPOINT one;  
  
Savepoint created.  
  
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(130, 'Estate Management', 102, 1500);  
  
1 row created.  
  
SQL>
```

Gambar 12.3 Penambahan data departemen baru ke dalam tabel
copy_departments

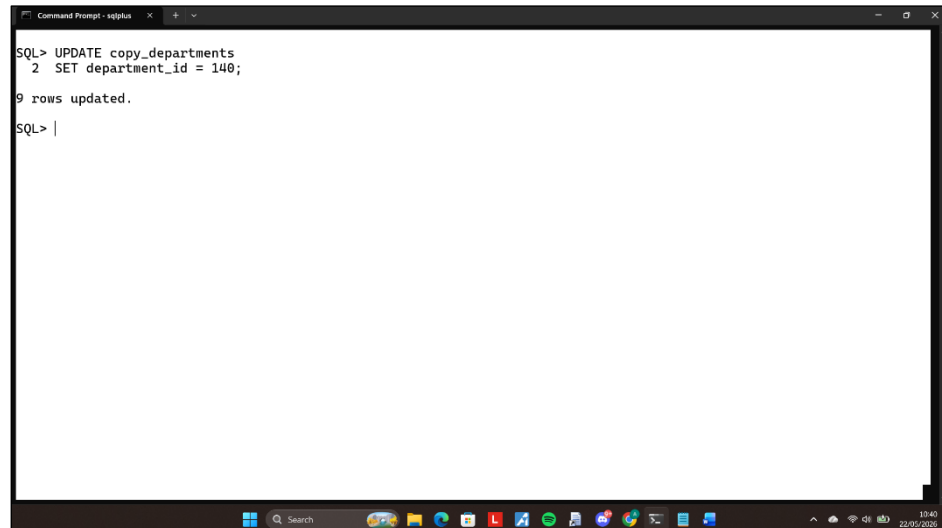
Penjelasan:

Query ini digunakan untuk menambahkan data baru ke dalam tabel copy_departments menggunakan perintah INSERT. Data yang dimasukkan meliputi department_id, department_name, manager_id, dan location_id secara lengkap. Proses ini merupakan bagian dari transaksi yang dapat dikontrol menggunakan TCL sebelum disimpan permanen ke *database*.

4. *Query*:

```
UPDATE copy_departments  
SET department_id = 140;
```

Hasil:

A screenshot of a Windows Command Prompt window titled "Command Prompt - sqlplus". The window shows the execution of an SQL UPDATE statement. The text displayed is: "SQL> UPDATE copy_departments", "2 SET department_id = 140;", "9 rows updated.", and "SQL> |". The window has a standard Windows taskbar at the bottom with various application icons and a system clock showing 10:40 on 22/11/2020.

```
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
9 rows updated.  
SQL> |
```

Gambar 12.4 Perubahan seluruh nilai `department_id` pada tabel `copy_departments`

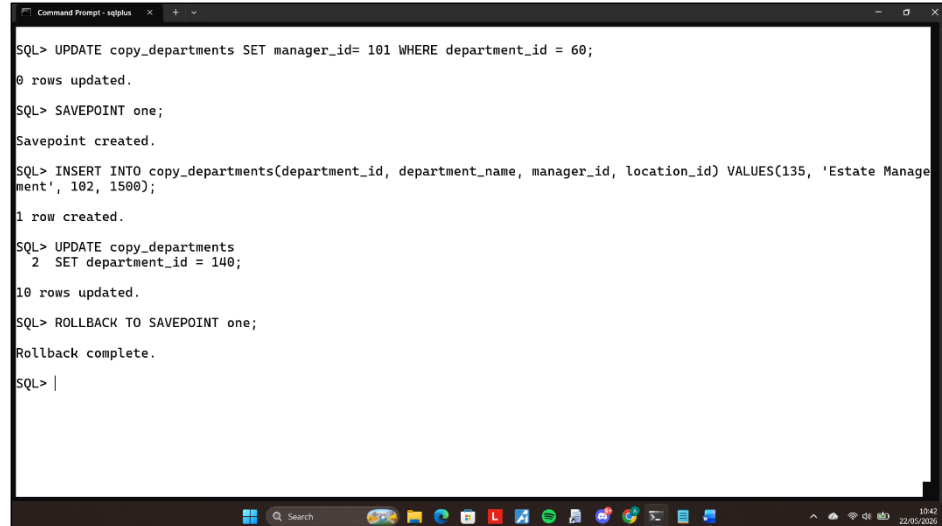
Penjelasan:

Query ini digunakan untuk memperbarui seluruh nilai pada kolom `department_id` menjadi 140 tanpa menggunakan kondisi `WHERE`. Hal ini menyebabkan semua baris dalam tabel mengalami perubahan secara menyeluruh. Operasi ini berisiko sehingga penting untuk dikontrol menggunakan `SAVEPOINT` atau `ROLLBACK` jika terjadi kesalahan.

5. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:



```
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;
0 rows updated.
SQL> SAVEPOINT one;
Savepoint created.
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(135, 'Estate Management', 102, 1500);
1 row created.
SQL> UPDATE copy_departments
2 SET department_id = 140;
10 rows updated.
SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
SQL> |
```

Gambar 12.5 Pembatalan transaksi menggunakan ROLLBACK ke
SAVEPOINT one

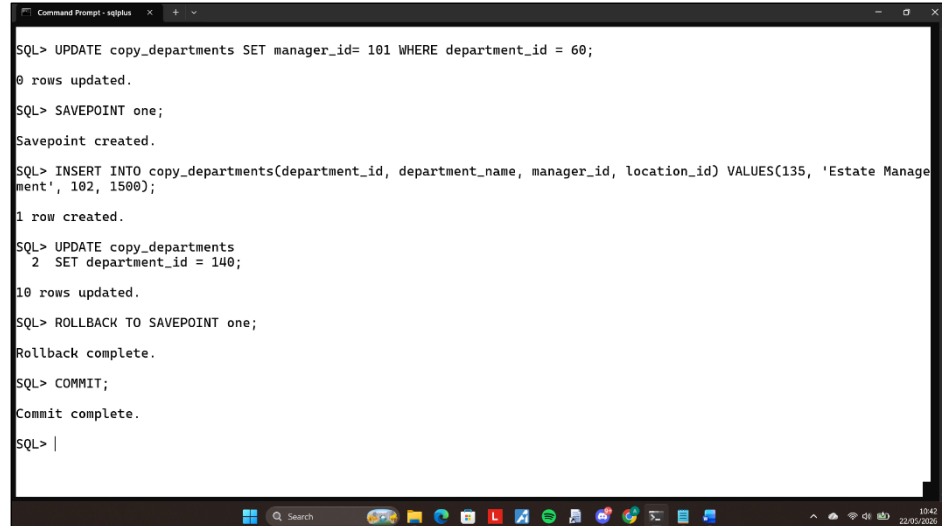
Penjelasan:

Query ini digunakan untuk membatalkan perubahan data hingga ke titik SAVEPOINT one yang telah dibuat sebelumnya. Perintah ROLLBACK memungkinkan pengguna mengembalikan kondisi data ke keadaan sebelum perubahan dilakukan. Dengan demikian, kesalahan dalam transaksi dapat diperbaiki tanpa harus mengulang seluruh proses dari awal.

6. *Query*:

```
COMMIT;
```

Hasil:



```
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;
0 rows updated.
SQL> SAVEPOINT one;
Savepoint created.
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(135, 'Estate Management', 102, 1500);
1 row created.
SQL> UPDATE copy_departments
2 SET department_id = 140;
10 rows updated.
SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
SQL> COMMIT;
Commit complete.
SQL> |
```

Gambar 12.6 Penyimpanan permanen transaksi ke dalam *database* menggunakan COMMIT

Penjelasan:

Query ini digunakan untuk menyimpan seluruh perubahan yang telah dilakukan dalam transaksi secara permanen ke *database*. Perintah COMMIT memastikan bahwa semua perubahan tidak dapat dibatalkan kembali. Setelah COMMIT dilakukan, data akan tetap tersimpan meskipun terjadi gangguan pada sistem.

12.4 Latihan

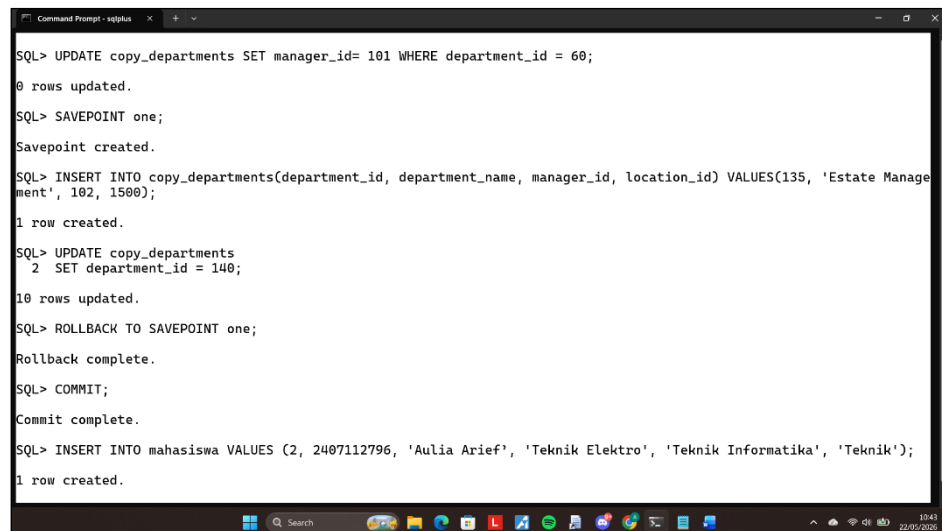
Latihan yang telah dilakukan pada praktikum 12 adalah sebagai berikut:

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO mahasiswa VALUES (2, 2407112796, 'Aulia Arief', 'Teknik Elektro', 'Teknik Informatika', 'Teknik');
```

Hasil:



```
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;
0 rows updated.
SQL> SAVEPOINT one;
Savepoint created.
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(135, 'Estate Management', 102, 1500);
1 row created.
SQL> UPDATE copy_departments
      2 SET department_id = 140;
10 rows updated.
SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
SQL> COMMIT;
Commit complete.
SQL> INSERT INTO mahasiswa VALUES (2, 2407112796, 'Aulia Arief', 'Teknik Elektro', 'Teknik Informatika', 'Teknik');
1 row created.
```

Gambar 12.7 Penambahan data mahasiswa ke tabel mahasiswa menggunakan perintah INSERT

Penjelasan:

Query ini digunakan untuk menambahkan data mahasiswa baru ke dalam tabel mahasiswa dengan informasi lengkap seperti NIM, nama, jenis kelamin, nomor telepon, dan program studi. Perintah INSERT memungkinkan penambahan data secara langsung ke dalam tabel sesuai struktur yang ada. Proses ini merupakan bagian dari transaksi yang dapat dikontrol menggunakan perintah TCL sebelum disimpan secara permanen.

12.5 Kesimpulan

Praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep *transaction control language* (TCL) dalam pengelolaan transaksi pada basis data Oracle. TCL berperan penting dalam mengatur bagaimana perubahan data yang dilakukan melalui perintah DML dapat disimpan atau dibatalkan sesuai kebutuhan. Dengan memahami konsep transaksi sebagai satu kesatuan proses, mahasiswa dapat mengelola data secara lebih terstruktur dan menghindari inkonsistensi dalam *database*.

Melalui penggunaan perintah `COMMIT`, mahasiswa dapat menyimpan seluruh perubahan data secara permanen sehingga hasil manipulasi data tidak hilang dan dapat digunakan kembali di kemudian hari. Sementara itu, perintah `ROLLBACK` memungkinkan pengguna untuk membatalkan perubahan yang telah dilakukan apabila terjadi kesalahan, sehingga kondisi data dapat dikembalikan seperti semula. Selain itu, penggunaan `SAVEPOINT` memberikan fleksibilitas lebih dalam transaksi karena memungkinkan `ROLLBACK` dilakukan hanya sampai titik tertentu tanpa membatalkan seluruh proses yang telah dikerjakan.

Secara keseluruhan, penerapan TCL sangat penting dalam menjaga konsistensi, keamanan, dan integritas data dalam suatu sistem basis data. Dengan menguasai perintah-perintah TCL, mahasiswa diharapkan mampu mengelola transaksi data secara efektif, mengantisipasi kesalahan dalam proses manipulasi data, serta menerapkan pengendalian transaksi yang baik dalam pengembangan sistem informasi yang lebih kompleks dan profesional.

DAFTAR PUSTAKA

- Budi, H. S. (2021). *Pengenalan dasar SQL*. Deepublish
- Oracle Corporation. (2026). *Hierarchical Queries*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/Hierarchical-Queries.html>
- Wade, D. (2023). *Mastering SQL joins: A quick handbook on mastering SQL joins with practical exercises*. dataceps.com.
- Wahidin, A. J., Miftachurohmah, N., Puryono, D. A., Hanes, Yunmar, R. A., Sembiring, S. B., Jayawardana, H., Kusuma, R. S., Hadi, H. N., Pardosi, I. A., Budiasto, J., & Rahayu, T. K. (2025). *Database*. Get Press Indonesia
- Oracle Corporation. (2026). *UPDATE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/26/sqlrf/UPDATE.html>
- Oracle Corporation. (2026). *CREATE TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-TABLE.html>
- Oracle Corporation. (2026). *DROP TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/DROP-TABLE.html>
- Oracle Corporation. (2026). *INSERT*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/INSERT.html>
- Oracle Corporation. (2026). *MERGE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/MERGE.html>
- Oracle Corporation. (2026). *USER*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/USER.html>
- Oracle Corporation. (2026). *SET TRANSACTION*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/SET-TRANSACTION.html>

Oracle Corporation. (2026). *COMMENT*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/COMMENT.html>

Oracle Corporation. (2026). *FLASHBACK TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/FLASHBACK-TABLE.html>

Oracle Corporation. (2026). *DROP TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/DROP-TABLE.html>

Oracle Corporation. (2026). *TRUNCATE TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/TRUNCATE-TABLE.html>

Oracle Corporation. (2026). *RENAME*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/RENAME.html>

Oracle Corporation. (2026). *ALTER TABLE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/ALTER-TABLE.html>

Oracle Corporation. (2026). *DATA TYPES*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/Data-Types.html>

Oracle Corporation. (2026). *USER*. Oracle Database SQL Language Reference.

Oracle Corporation. (2026). *CONSTRAINT*. Oracle Database SQL Language Reference. https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/constraint.html#GUID-1055EA97-BA6F-4764-A15F-1024FD5B6DFE__I1002349

Oracle Corporation. (2026). *CREATE VIEW*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-VIEW.html>

Oracle Corporation. (2026). *USING SUBQUERIES*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/Using-Subqueries.html>

- Oracle Corporation. (2026). *ROWNUM*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/ROWNUM-Pseudocolumn.html>
- Oracle Corporation. (2026). *CREATE SYNONYM*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-SYNONYM.html>
- Oracle Corporation. (2026). *CREATE SEQUENCES*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-SEQUENCE.html>
- Oracle Corporation. (2026). *CREATE INDEX*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/DROP-INDEX.html>
- Oracle Corporation. (2026). *DROP INDEX*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/CREATE-INDEX.html>
- Oracle Corporation. (2026). *GRANT*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/GRANT.html>
- Oracle Corporation. (2026). *REVOKE*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/REVOKE.html>
- Oracle Corporation. (2026). *TYPES OF SQL STATEMENTS*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/Types-of-SQL-Statements.html#GUID-FEC504E9-D22D-4082-A092-07891911C5CF>
- Oracle Corporation. (2026). *COMMIT*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/COMMIT.html>
- Oracle Corporation. (2026). *ROLLBACK*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/ROLLBACK.html>

Oracle Corporation. (2026). *SAVEPOINT*. Oracle Database SQL Language Reference. <https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/SAVEPOINT.html>